Principios de Desarrollo de software

Guía de Laboratorio Detallada: Implementación de un CRUD para un "Zoo de Mascotas Fantásticas" en Spring Boot

---

Objetivo del Laboratorio:

Este laboratorio tiene como objetivo desarrollar un sistema CRUD básico utilizando Spring Boot con Java, enfocándose en la creación de una arquitectura en 2 capas. Además, se integrará el uso de MySQL como base de datos y la gestión del proyecto con Git utilizando la estrategia Gitflow.

Descripción General del Proyecto: "Zoo de Mascotas Fantásticas"

Objetivo del Proyecto: Desarrollar un sistema que permita gestionar las criaturas mágicas en un zoológico, ofreciendo operaciones básicas de CRUD (Crear, Leer, Actualizar, Eliminar) sobre la base de datos de criaturas.

Funcionalidades Clave:

- **Crear Criatura:** Permitir la creación de una nueva criatura con atributos como nombre, especie, tamaño, nivel de peligrosidad y estado de salud.
- **Leer Información de Criaturas:** Mostrar una lista de todas las criaturas y permitir la visualización detallada de una criatura específica.
- **Actualizar Criatura:** Permitir la modificación de los atributos de una criatura existente.
- **Eliminar Criatura:** Eliminar una criatura, asegurando que no esté en estado de salud crítico.

Requerimientos Técnicos:

- **Capa de Acceso a Datos:** Implementar las operaciones CRUD utilizando Spring Data JPA con MySQL como base de datos.
- **Capa de Lógica de Negocio:** Gestionar las reglas de negocio, como validaciones y restricciones, antes de persistir los datos.
- **Gestión del Proyecto:** Uso de Git para el control de versiones, siguiendo la estrategia Gitflow.

Etapas del Proyecto:

1. **Investigación y preparación del entorno.**
    - Tener instalada una versión java 8 o superior.

- Realizar una investigación sobre lo que es **Maven**.
- Tener instalado Maven.
- Instalar un IDE para el desarrollo.

2. **Configuración del Proyecto:**
   - Crear un proyecto nuevo en Spring Boot utilizando Spring Initializr.
   - Configurar las dependencias necesarias: Spring Web, Spring Data JPA, MySQL Driver, y Lombok.
     - Estos son conceptos que se deben estudiar y deben ir en el marco teorico del informe.
   - Configurar Git y establecer la estrategia Gitflow para la gestión de ramas.
3. **Diseño de la Arquitectura:**
   - Definir una arquitectura en 2 capas, separando la lógica de negocio del acceso a datos.
4. **Implementación del CRUD:**
   - Crear la entidad `Creature`, el repositorio, el servicio y el controlador.
   - Implementar los endpoints necesarios para cada operación CRUD.
5. **Pruebas y Validación:**
   - Realizar pruebas manuales para verificar que el sistema cumple con los criterios de aceptación.
6. **Gestión de Versiones:**
   - Utilizar Gitflow para gestionar las ramas del proyecto, asegurando un flujo de trabajo colaborativo y organizado.

Implementación en Spring Boot con MySQL y Gitflow

1. Configuración del Proyecto:

- **Crear un Nuevo Proyecto:**
  - Acceder a Spring Initializr.
  - Configurar el proyecto:
    - Nombre del proyecto: `zoo-fantastico`
    - Lenguaje: Java
    - Dependencias: Spring Web, Spring Data JPA, MySQL Driver, Lombok

- **Configuración de MySQL:**
  - Aqui se tiene que crear la base de datos antes e implementar el servidor local para que cada uno lo tenga.

- En el archivo `application.properties`, configurar la conexión a MySQL:

```
spring.datasource.url=jdbc:mysql://localhost:3306/zoo_fantastico
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

- **Ajustar el repositorio de GIT.**
  - Todo el codigo debe quedar en git siguiendo gitflow

2. Creación de la Entidad `Creature`:

- **Definir la Entidad:**

```
import lombok.Data;
import lombok.NoArgsConstructor;
import javax.persistence.*;

@Entity
@Data // Esto es opcional pueden usarlo o crear los getter y setters
@NoArgsConstructor
public class Creature {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String species;
    private double size;
    private int dangerLevel;
    private String healthStatus;

    @ManyToOne
    @JoinColumn(name = "zone_id")
    private Zone zone;
}
```

```
•
```

3. Creación del Repositorio:

- **Definir el Repositorio `CreatureRepository`:**
  - Aqui es importante investigar y entender bien que es es jpa y porque es necesario crear esto.

```
public interface CreatureRepository extends JpaRepository<Creature, Long> {
}
```

4. Creación del Servicio `CreatureService`:

- **Implementar la Lógica de Negocio:**

```
@Service
public class CreatureService {
    private final CreatureRepository creatureRepository;

    @Autowired
    public CreatureService(CreatureRepository creatureRepository) {
        this.creatureRepository = creatureRepository;
    }

    public Creature createCreature(Creature creature) {
        return creatureRepository.save(creature);
    }

    public List<Creature> getAllCreatures() {
        return creatureRepository.findAll();
    }

    public Creature getCreatureById(Long id) {
        return creatureRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Creature not found"));
    }

    public Creature updateCreature(Long id, Creature updatedCreature) {
```

```
        Creature creature = getCreatureById(id);
        creature.setName(updatedCreature.getName());
        creature.setSpecies(updatedCreature.getSpecies());
        creature.setSize(updatedCreature.getSize());
        creature.setDangerLevel(updatedCreature.getDangerLevel());
        creature.setHealthStatus(updatedCreature.getHealthStatus());
        return creatureRepository.save(creature);
    }

    public void deleteCreature(Long id) {
        Creature creature = getCreatureById(id);
        if (!"critical".equals(creature.getHealthStatus())) {
            creatureRepository.delete(creature);
        } else {
            throw new IllegalStateException("Cannot delete a creature in critical health");
        }
    }
}
```

5. Creación del Controlador `CreatureController`:

- **Definir los Endpoints del CRUD:**

```
@RestController
@RequestMapping("/api/creatures")
public class CreatureController {
    private final CreatureService creatureService;

    @Autowired
    public CreatureController(CreatureService creatureService) {
        this.creatureService = creatureService;
    }

    @PostMapping
```

```
    public ResponseEntity<Creature> createCreature(@RequestBody Creature creature) {
        Creature newCreature = creatureService.createCreature(creature);
        return ResponseEntity.status(HttpStatus.CREATED).body(newCreature);
    }

    @GetMapping
    public List<Creature> getAllCreatures() {
        return creatureService.getAllCreatures();
    }

    @GetMapping("/{id}")
    public Creature getCreatureById(@PathVariable Long id) {
        return creatureService.getCreatureById(id);
    }

    @PutMapping("/{id}")
    public Creature updateCreature(@PathVariable Long id, @RequestBody Creature updatedCreature) {
        return creatureService.updateCreature(id, updatedCreature);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteCreature(@PathVariable Long id) {
        creatureService.deleteCreature(id);
        return ResponseEntity.noContent().build();
    }
}
```

Criterios de Aceptación:

1. **Crear Criatura:**
   - El sistema debe aceptar una nueva criatura con todos los campos válidos y persistirla en la base de datos MySQL.
   - **Validación:** No se permite que el `size` sea menor que 0, y el `dangerLevel` debe estar entre 1 y 10.
2. **Leer Información de Criaturas:**
   - Debe ser posible obtener una lista de todas las criaturas, y ver los detalles de una criatura específica por ID.
3. **Actualizar Criatura:**

   - La actualización de una criatura debe reflejarse en la base de datos, con validación de los datos actualizados.
4. **Eliminar Criatura:**
   - Solo debe ser posible eliminar una criatura si no está en estado de salud "critical".

Ahora que han implementado un CRUD para las criaturas mágicas, extender el proyecto para incluir un nuevo CRUD que gestione las "Zonas del Zoológico". Cada criatura debe estar asociada a una zona específica del zoológico, lo que requerirá una relación entre las entidades Creature y Zone.

- Incluir en el Informe, capturas de pantalla de las peticiones Http, Tip: Usar **postman** para esta parte.

Requerimientos:

- **Crear Zona:** Permitir la creación de nuevas zonas en el zoológico con atributos como name (nombre), description (descripción) y capacity (capacidad máxima de criaturas).
- **Leer Zonas:** Mostrar una lista de todas las zonas disponibles y la cantidad de criaturas asignadas a cada una.
- **Actualizar Zona:** Permitir la modificación de los atributos de una zona existente.
- **Eliminar Zona:** Permitir la eliminación de una zona, siempre y cuando no tenga criaturas asignadas.

Criterios de Aceptación:

- **Relación entre Zonas y Criaturas:** Asegurar que cada criatura esté asignada a una zona, y que al eliminar una zona, se verifique que no contiene criaturas antes de permitir la eliminación.

Gestión de Versiones con Git y Gitflow:

- **Rama `main`:** Contiene el código estable y listo para producción, hacer un PR final de develop a main en el cual deben colocar al profesor como revisor, Usuario github: soldash.
- **Rama `develop`:** Contiene el código en desarrollo, con nuevas características que se integran desde ramas feature.
- **Ramas `feature`**: Se utilizan para desarrollar nuevas características cada uno de los integantes del equipo, cada uno de los integrantes del equipo debe hacer una parte. Al finalizar, se integran por medio de Pull Requests a `develop`.

Tips para el Informe:

Documentar cómo implementaron el sistema, describiendo:

- Conceptos de la investigación de las herramientas.
- Cómo se planificaron y organizaron las tareas.
- Cómo se gestionó el progreso y la colaboración utilizando Git.
- Qué desafíos enfrentaron y cómo los superaron.
- **Descripción del Proyecto:** Detalle los objetivos, alcance y características clave del sistema CRUD.
- **Reflexión sobre la Experiencia:** Discuta la aplicabilidad de Spring